

Topic 3: Constructors and Destructors

3.1 What is a Constructor?

- A constructor is a special member function called automatically when an object is created
- Same name as the class, no return type (not even void)
- Used to initialise member variables to valid starting values
- If you don't write one, the compiler provides a default constructor that does nothing

```
class Rectangle {  
    double width, height;  
public:  
    Rectangle() {           // default constructor  
        width = 0; height = 0;  
    }  
    Rectangle(double w, double h) { // parameterised constructor  
        width = w; height = h;  
    }  
    double area() { return width * height; }  
};
```

3.2 Constructor Overloading

- You can define multiple constructors with different parameter lists -- this is overloading
- The compiler calls whichever constructor matches the arguments provided
- Default values in parameters can reduce the number of constructors needed

```
Rectangle r1;           // calls default constructor  
Rectangle r2(5.0, 3.0); // calls parameterised constructor  
Rectangle r3(4.0);      // works if height has a default value
```

3.3 Initialiser List

- Preferred way to initialise members: `ClassName::ClassName(args) : member(value) { }`
- Required for const members and reference members -- they cannot be assigned in the body
- More efficient than assignment inside the body -- avoids double initialisation

```
class Point {
    const int x, y;
public:
    Point(int a, int b) : x(a), y(b) {} // initialiser list
    void show() { cout << x << ", " << y << endl; }
};
```

3.4 Copy Constructor

- Called when an object is initialised from another object of the same class
- Signature: `ClassName(const ClassName& other)`
- Compiler provides a shallow copy by default -- copies values of members
- Write your own (deep copy) when the class contains pointers -- otherwise both objects share the same heap memory

```
Rectangle r1(6.0, 4.0);
Rectangle r2 = r1;    // copy constructor called
```

3.5 Destructor

- Called automatically when an object goes out of scope or `delete` is used on a heap object
- Same name as class, prefixed with `~`, no parameters, no return type
- Release resources: close files, free heap memory allocated in the constructor
- If you allocate memory with `new` in the constructor, always free it with `delete` in the destructor

```
class Buffer {
    int* data;
public:
    Buffer(int size) { data = new int[size]; }
    ~Buffer()        { delete[] data; }        // destructor
};
```